

Accountibuzz

Technical Report

Vision and Scope · Use Cases · Software Requirements Specification

A small-group accountability app for iOS and Android. Members submit daily photo or video proof of a personal commitment; a single group admin verifies each submission; points, streaks, and a live leaderboard turn verified work into visible social signal. This document captures the product vision, the canonical use cases, and the full software requirements specification for the v1.0 MVP.

Prepared by	Chris Kelamyan · Timothy Do
Document version	v1.0 — MVP scope
Last updated	May 8, 2026
Repository	accountibuzz
Source	docs/technical-report.md

Table of Contents

1. Vision and Scope Document	
1.1 Business Requirements	
1.1.1 Background	
1.1.2 Business Opportunity	
1.1.3 Business Objectives	
1.1.4 Success Metrics	
1.1.5 Vision Statement	
1.1.6 Business Risks	
1.1.7 Business Assumptions and Dependencies	
1.2 Scope and Limitations	
1.2.1 Major Features	
1.2.2 Scope of Initial and Subsequent Releases	
1.2.3 Limitations and Exclusions	
1.3 Business Context	
1.3.1 Stakeholder Profiles	
1.3.2 Project Priorities	
1.3.3 Deployment Considerations	
2. Use Cases	
UC-001 Sign Up, Sign In, and Complete Profile	
UC-002 Create a Group and Bring Friends In	
UC-003 Submit Daily Proof and Get It Reviewed	
UC-004 View Live Leaderboard and Group Feed	
UC-005 Receive Push Notifications and Trigger Daily Rollover	
3. Software Requirements Specification	
3.1 Introduction	
3.1.1 Purpose	
3.1.2 Document Conventions	
3.1.3 Project Scope	
3.2 Overall Description	
3.2.1 Product Perspective	

3.2.2 User Classes and Characteristics	
3.2.3 Operating Environment	
3.2.4 Design and Implementation Constraints	
3.2.5 Assumptions and Dependencies	
3.3 System Features	
3.3.1 Authentication and Profile	
3.3.2 Groups and Invites	
3.3.3 Capture and Admin Review	
3.3.4 Points, Streaks, Leaderboard, and Feed	
3.3.5 Notifications and Daily Rollover	
3.4 Data Requirements	
3.4.1 Logical Data Model	
3.4.2 Data Dictionary	
3.4.3 Reports	
3.4.4 Data Integrity, Retention, and Disposal	
3.5 External Interface Requirements	
3.5.1 User Interfaces	
3.5.2 Software Interfaces	
3.5.3 Hardware Interfaces	
3.5.4 Communication Interfaces	
3.6 Quality Attributes	
3.6.1 Usability Requirements	
3.6.2 Performance Requirements	
3.6.3 Security Requirements	
3.6.4 Safety Requirements	
3.6.5 Availability Requirements	
3.6.6 Robustness Requirements	

1. Vision and Scope Document

1.1 Business Requirements

1.1.1 Background

Most people who set personal goals miss them, and the failure mode is rarely a lack of information. People who want to work out three times a week already know how to work out, and people who want to write every day already know how to open a document. The piece that collapses is follow-through, and the cheapest fix in everyday life is having someone watching. A friend texting “did you go to the gym?” changes behavior in a way habit-tracking apps generally do not, because the cost of skipping stops being purely internal.

The existing market in this space splits into two camps. On one side sit habit trackers like Habitica and the various streak apps, which run on the honor system and treat the user as the only audience. On the other side sit commitment-contract products like StickK and Beeminder, which manufacture stakes by attaching money or an anti-charity to a missed goal. Neither model captures what actually works in real friend groups, where the stakes are social rather than financial and verification is human rather than algorithmic. Accountibuzz exists to fill that gap with a small, opinionated, mobile-first product.

1.1.2 Business Opportunity

The opportunity is a small-group, friend-shaped accountability app that takes the strict-streak mechanic seriously, requires daily photo or video proof, and routes verification through a single human admin. There is no public discovery, no leaderboard arms race, and no path to softening the streak with paid grace days. The bet is that a tightly-scoped product with one clear opinion will land with the people who already understand why social pressure works, and that those people will bring their own groups in via invite link rather than via marketing spend.

1.1.3 Business Objectives

ID	Objective	Measurable form
O-01	Validate that small-group social pressure plus media proof drives daily follow-through	At least three friend groups of 5–10 members complete two consecutive weeks with a median of four verified submissions per member per week
O-02	Ship a mobile MVP a small group of builders can maintain	One person can run discuss / plan / execute / verify per phase without external help, using the GSD workflow already in <code>.planning/</code>
O-03	Keep operating cost near zero through alpha	Stay inside Supabase + Expo + EAS free tiers for the duration of friend-group testing
O-04	Lock in trust-and-safety primitives before any external user	Every public-schema table has Row-Level Security, a CI workflow fails the build if that ever stops being true, and storage buckets are private with signed-URL access

ID	Objective	Measurable form
O-05	Preserve traceability from requirement to commit	Each requirement ID appears in <code>REQUIREMENTS.md</code> , the matching <code>ROADMAP.md</code> phase, the phase plan, the JIRA story, and the commit scope

1.1.4 Success Metrics

The metrics that matter for the MVP are group-level, not individual-level. A single user posting daily in a dead group is not the product working; a group of six holding 80% submission rate across two weeks is.

Metric	Target during friend-group alpha
Group weekly active rate (members submitting at least 5 days)	$\geq 70\%$ across week 2
Median admin time-to-review on a submission	≤ 6 hours
Submissions lost to upload failure	$\leq 1\%$ of attempts
Push delivery reliability across iOS and Android	$\geq 95\%$ deliveries within 60 seconds of trigger
Critical bugs found post-rollout that originate in RLS or auth	0
Day-14 group retention (a group still posting after two weeks)	$\geq 50\%$ of seeded groups

1.1.5 Vision Statement

For people who follow through when friends are watching but not when only they are, Accountibuzz is a mobile app that turns a small group into the audience. Every day, each member submits a short photo or video as proof of their commitment, an admin verifies it, and points and streaks update for everyone to see. Habit trackers run on the honor system. StickK-style products lean on money. Accountibuzz uses the social cost inside a tight friend group instead. A missed day is visible to the rest of the group within hours, and that visibility is the whole point.

1.1.6 Business Risks

ID	Risk	Likelihood	Impact	Mitigation
R-01	Week-2 death spiral – one member misses, others disengage, the group dies	High	High	Encourage 7–10 invitees for a target group of 5–6; ship a re-engagement prompt for broken-streak members in Phase 6; track group weekly active rate, not individual DAU
R-02	Admin becomes the bottleneck – submissions sit pending, the loop feels broken	Medium	High	Optimistic pending UI from Phase 3, swipe-stack review queue, push pings on each new submission, planned admin daily-digest in Phase 5
R-03	Strict streak reset feels punitive and drives churn	Medium	Medium	Communicate the rule explicitly at onboarding, design a re-entry prompt that does not violate the rule, hold the line on no-grace-days as a product opinion
R-04	Push notifications drop silently, especially on Android, breaking the daily prompt	Medium	High	Use Expo Push Service through <code>notifications_outbox</code> and a <code>push-dispatch</code> edge function, develop on EAS dev clients from day one, test on real Android devices before rollout
R-05	Media upload fails on flaky cellular and the user thinks they posted	High in test, medium in prod	High	Two-phase commit (upload then insert), explicit progress UI, AsyncStorage offline queue with retry, no silent success toasts
R-06	Row-level security misconfigured, anon key leaks data	Low if process holds	Critical	RLS enabled on every table by default, <code>rls-check.yml</code> CI workflow fails the build otherwise, pgTAP coverage of every policy
R-07	App store gating delays the move from Expo Go to a dev build with push entitlements	Medium	Medium	EAS credentials and the dev-build cutover are scheduled into Phase 5 with lead time built in

ID	Risk	Likelihood	Impact	Mitigation
R-08	Solo-builder bus factor of one	Low during alpha	Medium	All planning, plans, verification, and decisions are checked into <code>.planning/</code> ; future contributors can read the project from the repo alone

1.1.7 Business Assumptions and Dependencies

Assumptions about user behavior:

- Small groups of 5–10 people who already know each other will use a shared app to hold one another accountable
- Those groups will accept a single-admin verification model rather than demand democratic or rotating review
- A daily submission cadence will not burn groups out

Assumptions about the platform:

- Supabase remains a viable single-platform backend (auth, Postgres, storage, realtime, edge functions, cron) through alpha
- Expo SDK 55 stays on its current release line without forcing a migration mid-build
- Apple and Google continue to leave Expo's push-notification path intact

External dependencies:

- Friend groups are willing to install a TestFlight or Play Internal build before any App Store submission
- The timezone database shipped with Postgres remains the source of truth for “what local date is it for this group right now”

1.2 Scope and Limitations

1.2.1 Major Features

The MVP covers eight feature areas, each owned by a phase in the roadmap.

#	Feature area	What it delivers
1	Authentication	Email-and-password sign-up via Supabase Auth; session persists across restarts (SecureStore-encrypted)
2	Groups	Create a group with name, goal, submission type (photo or video), and IANA timezone; creator becomes admin
3	Invites	Admin mints a shareable deep link or 8-character code; recipient joins with one tap
4	Submissions	In-app camera capture, two-phase upload with offline queue, server-derived local date that prevents backdating

#	Feature area	What it delivers
5	Admin Review	Swipe-style queue with approve / reject actions, optional reject reason, terminal-rejection semantics
6	Points & Streaks	Server-side trigger off the approval event: 1 point per verified day, streak increments on consecutive verified local dates, resets to zero on any miss
7	Leaderboard & Group Feed	Points/streak ranking that updates in near real time over Supabase Realtime; daily feed showing approved submissions, still-to-post members, and yesterday's-miss tombstones
8	Push Notifications & Daily Rollover	Pre-deadline reminder, activity pings on submission and review events, <code>pg_cron</code> rollover at each group's local midnight (DST-safe)

1.2.2 Scope of Initial and Subsequent Releases

Release	Phase	Scope
v1.0 (alpha to friend groups)	Phase 1 – Foundation	Expo SDK 55 scaffold, Supabase client with SecureStore-encrypted session, full Postgres schema with RLS on every table, theme tokens and primitives, four auth screens, profile screen with avatar upload
v1.0	Phase 2 – Groups & Invites	Create-group form, IANA timezone picker, 7 SECURITY DEFINER RPCs (create/redeem/preview/leave/transfer/delete/regenerate), groups list, group detail with invite chip and members, join-by-code screen, deep-link landing
v1.0	Phase 3 – Capture & Admin Review	Capture screen with state machine, two-phase commit upload pipeline, AsyncStorage offline queue, admin swipe-stack review queue with reject-reason panel, terminal-rejection semantics
v1.0	Phase 4 – Social Surfaces (active)	Points/streak triggers, leaderboard with Realtime updates, group feed with approved submissions, still-to-post avatar row, missed-yesterday tombstones
v1.0	Phase 5 – Push & Daily Rollover	EAS dev/production build, push entitlements on iOS and Android, <code>notifications_outbox</code> , <code>push-dispatch</code> edge function, <code>pg_cron</code> rollover job
v1.0	Phase 6 – Pre-Rollout Hardening	Re-entry prompt for broken streaks, admin group-health view, onboarding nudge to invite more than the target size, final RLS audit
v1.x	Post-MVP wins (FEED-adjacent)	Reactions on submissions, submission history per member, backup admin role

Release	Phase	Scope
v2.0	Retention & re-engagement	RE-01..04 — instrumented response time, week-1/week-2 retention dashboards, structured re-entry ritual, broader onboarding work
v2.0+	Differentiators	Public group discovery, web companion, monetization or premium groups

1.2.3 Limitations and Exclusions

The project deliberately excludes a number of features that look obvious but actively fight the core opinion.

Excluded	Reason
Public group discovery	Invite-only leverages existing social trust and avoids a moderation surface
Streak freezes / grace days / configurable strictness	Softening the streak kills the social pressure that makes the product work
Scaled point values / milestone bonuses	Keeps the leaderboard legible; revisit only with real group-behavior data
Rotating admins, peer voting, AI auto-verification	The human admin is the trust anchor — that is the differentiation
Auto-approval after a timeout	Same reason; pending-until-reviewed keeps the human in the loop
Custom per-group cutoff times	One less knob for groups to misconfigure
In-app chat or DMs	Friend groups already have group chats; conflicts with the quiet positioning
Camera-roll uploads as primary path	Proof credibility depends on in-the-moment capture
Web or desktop client	Daily capture is a phone-native behavior
Monetization / subscriptions	Out of scope for the MVP entirely
Android-specific UX work beyond Expo defaults	iOS is the leading surface during alpha

1.3 Business Context

1.3.1 Stakeholder Profiles

Stakeholder	Major value	Attitudes	Major interests	Constraints
Group Member	Stays accountable to a personal goal because friends are watching	Wants the app out of the way after a fast capture; cares deeply that streaks are correct	Reliable upload, visible feed, fair streak math	One-handed phone use, flaky cellular, varied iOS/Android device mix
Group Admin	Shapes the group's standards by deciding what counts	Will tolerate review work if it stays quick; resents friction	Fast review queue, clear pending count, ability to reject with a reason	Time-constrained; reviews in 2-minute bursts
Project Owner / Solo Builder	Validates the social-pressure thesis before scaling effort	Pragmatic; willing to ship narrow scope	Clean traceability, low ops cost, ability to maintain alone	One person for engineering, design, and PM
Friend-Group Alpha Tester	Provides the first real-world signal	Forgiving on polish, harsh on broken streaks	A working product that respects their time	Real devices, real network conditions, real social context
Course Instructor (COMP586)	Evaluates process and engineering rigor	Reviews against established SE practice	Documentation, traceability, demonstrable verification	Academic deadlines
Future Contributor	Picks up the codebase	Reads <code>.planning/</code> and <code>docs/processes/</code> first	Self-contained context	Context lives in repo, not in heads

1.3.2 Project Priorities

Dimension	Priority	Comment
Schedule	High	Friend-group alpha is the goal of this release cycle; calendar is the binding constraint
Quality of the core loop (capture → review → streak)	Critical	If this breaks, nothing else matters
Quality of peripheral surfaces	Medium	Polish on profile, leaderboard animations, etc. can iterate after alpha
Feature breadth	Low	Scope is locked; new ideas land in <code>.planning/notes/</code> for v2

Dimension	Priority	Comment
Cost	High	Stay inside free tiers
Staff (one person)	Fixed	Solo builder is a hard constraint

The trade-off rule the project follows is that scope and quality on the core loop are non-negotiable, schedule absorbs the slack, and peripheral polish is the first thing cut if a phase runs long.

1.3.3 Deployment Considerations

Mobile client distribution:

- Targets iOS and Android through Expo SDK 55, with the New Architecture enabled (the only option in 55)
- Runs in Expo Go on iOS through Phase 4 for fast iteration
- Phase 5 forces the cutover to an EAS development build (`expo-notifications` no longer registers a push token in Expo Go on Android, and has been dev-client-only on iOS for several SDKs)
- iOS alpha distribution: TestFlight via `eas submit --profile preview`
- Android alpha distribution: Play Internal Testing or a direct APK from `eas build --profile preview --platform android`
- App Store and Play Store submission are explicitly post-alpha

Backend deployment:

- All SQL lives in `supabase/migrations/`, append-only once applied, deployed via `supabase db push`
- Edge functions live in `supabase/functions/` and ship via `supabase functions deploy`
- RLS is enforced on every public-schema table; `rls-check.yml` (GitHub Actions) fails the build if a new table ever ships with `rowsecurity = false`
- Local dev: `supabase start` for the stack, `supabase test db` for pgTAP
- `supabase gen types typescript --local > src/types/database.ts` keeps the TypeScript client in sync with the schema

2. Use Cases

The use cases below describe the canonical user-visible flow for each implementation phase of the v1 MVP roadmap. Each use case is keyed to a single phase in `.planning/ROADMAP.md` and traces to the requirement IDs that phase owns in `.planning/REQUIREMENTS.md`. Phase 6 (Pre-Rollout Hardening) owns no v1 user-facing requirement and is not represented here.

UC	Phase	Title	Owns requirements
UC-001	1 – Foundation	Sign Up, Sign In, and Complete Profile	AUTH-01..04, PLAT-01, PLAT-02
UC-002	2 – Groups & Invites	Create a Group and Bring Friends In	GRP-01..05, INV-01..03
UC-003	3 – Capture & Admin Review	Submit Daily Proof and Get It Reviewed	SUB-01..06, ADM-01..04, PLAT-03
UC-004	4 – Social Surfaces	View Live Leaderboard and Group Feed	PTS-01..03, LB-01..02, FEED-01..03
UC-005	5 – Push & Daily Rollover	Receive Push Notifications and Trigger Daily Rollover	NOTIF-01..05, ROLL-01..02

UC-001 Sign Up, Sign In, and Complete Profile

Field	Content
ID and Name	UC-001 Sign Up, Sign In, and Complete Profile
Created By	Chris Kelamyan, Timothy Do
Date Created	May 8, 2026
Primary Actor	New User (becomes Registered User)
Secondary Actors	System, Email Provider (password reset / verification)
Description	This use case describes how a new user signs up for AccountiBuzz with an email and password, persists their session across app restarts, signs out, and completes a profile with display name and avatar. Phase 1 of the roadmap establishes the identity layer that every other use case depends on.

Field	Content
Trigger	The user opens the app for the first time (no persisted session) or after a sign-out.
Preconditions	PRE-1. The app is installed (Expo build) on the user's device. PRE-2. The Supabase backend is reachable. PRE-3. The email provider is operational for password-reset and verification messages.
Postconditions	POST-1. A row exists in <code>auth.users</code>. POST-2. A <code>profiles</code> row is auto-created via the <code>handle_new_user</code> trigger. POST-3. The session is persisted in encrypted AsyncStorage (256-bit AES key wrapped in SecureStore). POST-4. The user is signed in and lands on the groups-list screen.
Normal Flow	1. The user opens the app and taps "Sign up." 2. The system displays the sign-up form. 3. The user enters email and password. 4. The system validates inputs (Zod client-side; Supabase Auth server-side). 5. The system creates the <code>auth.users</code> row. 6. The system creates the corresponding <code>profiles</code> row via the <code>AFTER INSERT</code> trigger. 7. The system persists the session in encrypted local storage. 8. The system routes the user to the profile-onboarding screen. 9. The user enters a display name. 10. The user optionally uploads an avatar. 11. The system uploads the avatar to the <code>avatars</code> storage bucket at <code>{user_id}/avatar.jpg</code>. 12. The system updates the <code>profiles</code> row with <code>display_name</code> and <code>avatar_path</code>. 13. The system routes the user to the groups list (empty state on first run).
Alternative Flows	A1. Existing user signs in 1. On the welcome screen the user taps "Sign in." 2. The user enters email and password. 3. Supabase Auth returns a session. 4. The session is persisted; the user lands on the groups list with their existing memberships visible. A2. App restart with a valid session 1. The app reads the encrypted session blob from AsyncStorage at launch. 2. The Supabase client rehydrates the session without prompting for credentials. 3. The user lands directly on the groups list. A3. User signs out 1. From the profile screen the user taps "Sign out." 2. The system clears the persisted session and the React Query cache. 3. The system routes the user back to the welcome screen. A4. Forgotten password 1. From the sign-in screen the user taps "Forgot password." 2. The user enters their email. 3. Supabase Auth sends a reset email. 4. The user taps the deep link, which opens the reset-password screen. 5. The user enters a new password and the system updates it.

Field	Content
Exceptions	E1. Email already registered 1. The user submits a sign-up form for an email tied to an existing account. 2. Supabase Auth returns an “email already registered” error. 3. The system displays an inline error and offers a sign-in shortcut. E2. Network unavailable during sign-up 1. The user submits the sign-up form. 2. The Supabase request times out. 3. The system displays “Unable to reach the server” and keeps the form filled in. E3. Avatar upload fails 1. The user selects an avatar. 2. The upload to storage fails (file too large, network drop). 3. The system surfaces an error and lets the user retry or skip the avatar; the rest of the profile still saves.
Priority	Highest
Frequency of Use	Sign-up: once per user. Sign-in / session restore: daily. Profile edit: rare.
Business Rules	BR-1. Only registered, signed-in users can access any other use case. BR-2. Each user has exactly one <code>auth.users</code> row and exactly one <code>profiles</code> row. BR-3. Sessions persist across app restarts via the SecureStore-encrypted AsyncStorage adapter. BR-4. Display name is mandatory; avatar is optional. BR-5. Sign-out clears all client cache and persisted state.
Special Requirements	SR-1. All client-server traffic is HTTPS. SR-2. Every public-schema table has Row-Level Security enabled (PLAT-02), enforced in CI by <code>rls-check.yml</code>. SR-3. The session blob in AsyncStorage is encrypted with a 256-bit AES key stored in SecureStore. SR-4. Sign-up to home-screen completes in under 5 seconds on a typical network.
Assumptions	• Email + password is sufficient for the alpha audience; OAuth, magic-link, and phone OTP are out of scope for v1. • Email delivery for password resets is reliable enough for alpha. • iOS and Android share the same auth flow.

UC-002 Create a Group and Bring Friends In

Field	Content
ID and Name	UC-002 Create a Group and Bring Friends In
Created By	Chris Kelamyan, Timothy Do
Date Created	May 8, 2026
Primary Actor	Registered User (becomes Group Admin)

Field	Content
Secondary Actors	System, Invitee (becomes Group Member), Operating System (share sheet, deep links)
Description	This use case describes how a registered user creates a group with a name, goal, submission type (photo or video), and IANA timezone, becomes its admin, and brings friends in via a shareable link or 8-character code. Phase 2 of the roadmap delivers the container that every submission depends on.
Trigger	A registered user taps “New group” on the groups-list screen.
Preconditions	PRE-1. The user is registered and signed in (UC-001). PRE-2. A <code>profiles</code> row exists for the user. PRE-3. The Supabase backend is reachable.
Postconditions	POST-1. A <code>groups</code> row exists with the user as <code>admin_user_id</code> . POST-2. A <code>group_members</code> row exists for the creator with <code>role='admin'</code> . POST-3. An <code>invites</code> row exists with a fresh 8-character code (31-character ambiguity-stripped alphabet) and a 7-day expiry. POST-4. The user lands on the group-detail screen with the invite chip ready to share.
Normal Flow	1. The user opens the groups list and taps “New group.” 2. The system displays the create-group form. 3. The user enters a group name (1–60 characters). 4. The user enters a goal description (5–140 characters). 5. The user selects a submission type: photo or video. 6. The user opens the timezone picker and selects an IANA timezone (defaults to the device’s current zone). 7. The user taps “Create group.” 8. The system validates inputs client-side via Zod and server-side via typed RPC errors. 9. The system invokes the <code>create_group</code> RPC, which atomically inserts the group row, the admin membership row, and the first invite under one transaction. 10. The system routes the user to the group-detail screen. 11. The user taps the invite chip to share. 12. The OS share sheet opens; the user picks a destination (Messages, WhatsApp, etc.). 13. The recipient taps the deep link <code>accountibuzz://invite/<CODE></code> (or the universal-link equivalent). 14. The system fetches a 3-field preview (group name, member count, admin display name) via <code>get_invite_preview</code>. 15. The recipient taps “Join group.” 16. The system invokes <code>redeem_invite</code>, which row-locks the invite and the group, validates capacity, inserts the membership row, and stamps the invite as used. 17. The recipient lands on the group-detail screen as a member.

Field	Content
Alternative Flows	A1. Recipient does not have the app installed 1. The universal link routes through the App Store / Play Store. 2. The recipient installs the app. 3. The pending invite code is captured and stored in SecureStore during the auth detour. 4. After sign-up (UC-001), <code>usePendingInviteReplay</code> redeems the invite automatically. 5. The user lands on the group-detail screen. A2. Recipient pastes the code instead of tapping a link 1. The recipient opens the app and taps “Join by code” on the groups list. 2. The recipient enters the 8-character code. 3. The system fetches the invite preview and displays it. 4. The recipient confirms; <code>redeem_invite</code> proceeds as in the normal flow. A3. Admin regenerates the invite code 1. From the group-detail screen the admin taps “Regenerate invite.” 2. The system calls <code>regenerate_invite</code>, which marks any existing active invite as used and mints a new code. 3. The admin sees the new code; old codes immediately stop working. A4. Member leaves the group 1. From the group-detail screen the member taps “Leave group.” 2. The system calls <code>leave_group</code>, which removes the membership row. 3. The member returns to the groups list with the group removed. A5. Admin transfers admin role 1. From the group-detail screen the admin selects another member and taps “Transfer admin.” 2. The system calls <code>transfer_admin</code>, which atomically updates <code>groups.admin_user_id</code> and both members’ role columns inside one transaction. 3. The original admin is now a member; the new admin sees the admin-only controls.
Exceptions	E1. Group cap reached (10 members) 1. An invitee attempts to redeem for a full group. 2. <code>redeem_invite</code> raises <code>group_full</code> under the row lock on the groups row (preventing TOCTOU). 3. The system displays “This group is full.” E2. Invite expired or already used 1. The recipient attempts to redeem. 2. <code>redeem_invite</code> raises <code>invite_expired</code> or <code>invite_already_used</code>. 3. The system displays an explanatory message and offers to ask the admin for a new link. E3. Recipient is already a member 1. The recipient taps an invite link for a group they’re already in. 2. <code>redeem_invite</code> raises <code>already_member</code>. 3. The system routes them to the existing group-detail screen. E4. Invalid or unknown code 1. The recipient enters or taps an invalid code. 2. <code>redeem_invite</code> and <code>get_invite_preview</code> raise a uniform <code>invite_not_found</code> regardless of cause, so attackers cannot enumerate state. 3. The system displays “We couldn’t find that invite.” E5. Admin attempts to leave their own group 1. The admin taps “Leave group.” 2. <code>leave_group</code> raises <code>admin_cannot_leave</code>. 3. The system explains they must transfer admin first or delete the group.

Field	Content
Priority	Highest
Frequency of Use	Group create: occasional. Invite redemption: spike at group formation, then rare. Leave / transfer: rare.
Business Rules	BR-1. The group creator is automatically assigned as the admin. BR-2. Each group has exactly one admin at any time. BR-3. A group is capped at 10 members (soft cap enforced server-side under row lock). BR-4. Submission type (photo or video) is set at creation and not changed afterward. BR-5. Group timezone is an IANA identifier, not an offset, so DST is correct. BR-6. Only one active invite code exists per group at any time; regenerating closes the prior one. BR-7. Invites expire after 7 days by default. BR-8. Admins cannot leave a group; they must transfer admin first.
Special Requirements	SR-1. Invite codes are 8 characters from a 31-character ambiguity-stripped alphabet (no <code>0</code> , <code>0</code> , <code>1</code> , <code>I</code> , <code>L</code>); approximately 8.5×10^{11} possibilities. SR-2. <code>get_invite_preview</code> returns a uniform <code>invite_not_found</code> for any non-matching code to prevent enumeration. SR-3. All RPCs are <code>SECURITY DEFINER</code> with <code>set search_path = public</code> and typed error codes. SR-4. Admin-only RPCs (<code>regenerate_invite</code> , <code>transfer_admin</code> , <code>delete_group</code>) verify admin status server-side against <code>groups.admin_user_id</code> .
Assumptions	- Recipients have iMessage, WhatsApp, or SMS for sharing; the OS share sheet handles the rest. - The 10-member cap is sufficient for the alpha audience. - Universal-link domain configuration is in place by the time external alpha begins.

UC-003 Submit Daily Proof and Get It Reviewed

Field	Content
ID and Name	UC-003 Submit Daily Proof and Get It Reviewed
Created By	Chris Kelamyan, Timothy Do
Date Created	May 8, 2026
Primary Actor	Group Member
Secondary Actors	Group Admin, System, Camera, Storage, Database
Description	This use case describes how a group member captures a photo or video as proof of their daily commitment, uploads it through a two-phase commit pipeline, and waits for the group admin to approve or reject. The local date the submission counts for is derived server-side from the group's IANA timezone. Rejection is terminal – the member does not get to resubmit that day. Phase 3 delivers the core accountability loop.
Trigger	A group member opens the Today screen and taps “Capture” before the group's local-midnight cutoff.

Field	Content
Preconditions	PRE-1. The user is signed in and a member of the group. PRE-2. The group has a defined timezone and submission type (set in UC-002). PRE-3. The current time in the group's timezone is before midnight. PRE-4. The user has not yet submitted for today's local date. PRE-5. Camera permission is granted (or will be requested).
Postconditions	POST-1. A <code>submissions</code> row exists with <code>status='pending'</code>, the correct <code>local_date</code>, the <code>media_path</code>, and any caption. POST-2. A media object exists at <code>submissions/{group_id}/{user_id}/{timestamp}.{ext}</code> in the private storage bucket. POST-3. The Today screen shows the user's pending status pill. POST-4. After admin review the row's <code>status</code> is <code>approved</code> or <code>rejected</code>, with <code>reviewed_by</code> and <code>reviewed_at</code> set, and <code>rejection_reason</code> set when rejected.
Normal Flow	1. The member opens the Today screen and sees the group's submission type and the time-to-deadline countdown. 2. The member taps "Capture." 3. The system requests camera permission if not yet granted. 4. The capture screen opens with the camera preview live and the shutter button visible. 5. The member captures the photo (single tap) or records the video (≤ 30 seconds). 6. The member optionally enters a caption (≤ 140 characters). 7. The member taps "Submit." 8. The system uploads the media to the private <code>submissions</code> bucket using the two-phase commit pattern; the upload progresses with a visible progress indicator. 9. The system invokes the <code>submit_today</code> RPC with <code>group_id</code>, <code>media_path</code>, <code>media_type</code>, and caption. 10. The RPC verifies group membership, validates <code>media_type</code> against the group's <code>submission_type</code>, derives <code>local_date</code> from <code>(now() AT TIME ZONE groups.timezone)::date</code>, and inserts the submissions row. 11. The system displays the pending status pill on the Today screen. 12. The admin's device receives a Realtime pending-count update and a push notification (Phase 5). 13. The admin opens the review-queue tab on the group. 14. The system fetches the queue via <code>get_pending_review_queue</code> (admin-gated RPC, not direct table read). 15. For each card the admin sees the submitter, media, caption, and timestamp. 16. The admin swipes right or taps "Approve"; the system calls <code>review_submission</code> with <code>decision='approved'</code>. 17. The submission row's status flips to <code>approved</code>; the trigger increments the submitter's points and <code>current_streak</code> (Phase 4) and writes a <code>notifications_outbox</code> row for the submitter (Phase 5). 18. The submitter sees their status pill update from "Pending" to "Approved" via Realtime and receives a push notification.

Field	Content
Alternative Flows	A1. Admin rejects with a reason 1. The admin swipes left or taps "Reject." 2. The reject-reason panel opens. 3. The admin optionally enters a reason (≤ 140 characters). 4. The admin confirms. 5. The system calls <code>review_submission</code> with <code>decision='rejected'</code> and the reason. 6. Submission status flips to <code>rejected</code>; the member cannot resubmit for that local date (decision D-12 – rejection is terminal). A2. Member adds a caption only after capture 1. After capturing the member focuses the caption field. 2. The member types a caption. 3. The submit flow continues normally with the caption included. A3. Member backgrounds the app mid-upload 1. The user switches apps while the upload is in progress. 2. The upload-queue manager keeps the operation alive. 3. On return the user sees the same progress state and (typically) the upload completing.
Exceptions	E1. Network drops during upload 1. The storage upload fails partway. 2. The upload-queue manager persists the intent (local file URI + metadata) to AsyncStorage. 3. On reconnect the queue retries the upload. 4. No <code>submissions</code> row is inserted until the storage upload succeeds, so there is no orphan row. E2. Member already submitted today 1. The member taps "Submit" for a day they have already submitted. 2. The unique <code>(group_id, user_id, local_date)</code> constraint rejects the insert. 3. <code>submit_today</code> re-raises this as a typed <code>already_submitted_today</code> error. 4. The system displays "You've already submitted today." E3. Camera permission denied 1. The system prompts for camera permission. 2. The user denies it. 3. The capture screen shows a permission-denied state with a deep link to OS settings. E4. Cross-group attack on review 1. An admin of group A attempts to call <code>review_submission</code> with a <code>submission_id</code> belonging to group B. 2. <code>review_submission</code> re-resolves <code>group_id</code> from the database (not from client input) and checks <code>is_group_admin</code> against that <code>group_id</code>. 3. The call is rejected with <code>not_admin</code>. E5. Two admins approve concurrently 1. Two admin devices tap Approve on the same submission within seconds. 2. The first call acquires the row lock and updates status to <code>approved</code>. 3. The second call's <code>where status = 'pending'</code> clause matches zero rows. 4. The second call raises <code>not_pending</code>; the second admin sees an "Already reviewed" message. E6. Non-admin deep-links to the review screen 1. A non-admin member opens <code>/groups/[id]/review</code> directly. 2. The screen's <code>useGroup(groupId).is_admin</code> check redirects them. 3. If they bypass the client check, <code>get_pending_review_queue</code> raises <code>not_admin</code> server-side.
Priority	Highest

Field	Content
Frequency of Use	Submit: daily per member. Review: bursty per admin (multiple times per day during friend-group alpha).
Business Rules	BR-1. Exactly one submission per <code>(group_id, user_id, local_date)</code> is allowed (DB unique constraint). BR-2. Submissions are <code>pending</code> until the admin reviews them. BR-3. Rejection is terminal – the member does not get to resubmit that day. BR-4. <code>local_date</code> is derived server-side from the group's IANA timezone; the client never sends it. BR-5. Only the group's admin can review submissions (RLS + RPC double-gate). BR-6. Admins cannot impersonate submissions on behalf of other members. BR-7. Video submissions are capped at 30 seconds at capture time. BR-8. Media is stored in a private bucket; reads require a signed URL gated by group membership.
Special Requirements	SR-1. Upload uses a two-phase commit (storage first, then row insert) with cleanup of orphans by a planned scheduled job. SR-2. Upload survives airplane-mode-on-submit via the AsyncStorage offline queue with explicit retry. SR-3. The pending status pill must appear immediately when the user taps Submit; admin lag does not delay the submitter's UI. SR-4. Submission insert is atomic with timezone-derived <code>local_date</code>; race conditions on concurrent retries are absorbed by the unique constraint. SR-5. Admin review queue is paginated to 50 oldest-pending rows and gated by an admin-only RPC (defense-in-depth: RLS + RPC + client check).
Assumptions	- Members capture in-app, not from the camera roll, by default – proof credibility depends on in-the-moment capture. - Single admin per group is acceptable for alpha; revisit with real data if admin response time becomes a bottleneck. - Video, when chosen, stays short enough to upload on cellular within the SLA.

UC-004 View Live Leaderboard and Group Feed

Field	Content
ID and Name	UC-004 View Live Leaderboard and Group Feed
Created By	Chris Kelamyan, Timothy Do
Date Created	May 8, 2026
Primary Actor	Group Member
Secondary Actors	System, Database, Realtime channel

Field	Content
Description	This use case describes how a group member opens a group's detail screen and sees a leaderboard ranked by points and streak, plus a daily feed showing today's approved submissions, members who have not yet submitted, and tombstones for yesterday's misses. Both leaderboard and feed update in near real time as the admin approves submissions. Phase 4 turns verified work into visible social signal.
Trigger	A group member taps a group on the groups-list screen.
Preconditions	PRE-1. The user is signed in and a member of the group. PRE-2. The Phase 4 trigger that increments points and streak on approval is live. PRE-3. Realtime is enabled on <code>submissions</code> and <code>group_members</code> .
Postconditions	POST-1. The group-detail screen is rendered with leaderboard, today's feed, still-to-post avatars, and missed-yesterday tombstones. POST-2. A Realtime channel filtered on <code>group_id=eq.<id></code> is subscribed. POST-3. When an admin approves a submission, the leaderboard reorders and the feed updates without a manual refresh.
Normal Flow	1. The member taps a group on the groups list. 2. The system fetches the group's metadata, members, and current leaderboard via an RLS-gated query against <code>group_members</code>. 3. The system fetches today's approved submissions via a query against <code>submissions</code> filtered by <code>group_id</code> and today's <code>local_date</code>. 4. The system computes the still-to-post avatar row by diffing members against today's submitters. 5. The system fetches yesterday's misses (members with no approved submission for yesterday's <code>local_date</code>) for the tombstone strip. 6. The system subscribes to the Realtime channel for this group on <code>submissions</code> (INSERT/UPDATE) and <code>group_members</code> (UPDATE). 7. The screen renders four sections in this order: Leaderboard → Today's posts → Still to post → Missed yesterday. 8. The member sees ranks, their own row labeled "(you)", flame-emoji streak counts in tabular numerals, and the points number in extra-bold tabular type. 9. When the admin approves a submission elsewhere: a. The <code>submissions</code> UPDATE Realtime event fires. b. The trigger updates <code>group_members.points</code> and <code>current_streak</code>; the <code>group_members</code> UPDATE Realtime event fires. c. The Realtime handler patches the TanStack Query cache; the leaderboard reorders smoothly and the new feed item appears. d. The still-to-post row removes the submitter's avatar. 10. The member tapping a feed item opens the media at full size (private bucket; signed URL).

Field	Content
Alternative Flows	A1. Member opens the leaderboard expanded view 1. The member taps “See full leaderboard.” 2. The system renders the full ranked list inline (no modal) with all members visible. A2. Member backgrounds the app 1. The user switches apps; the Realtime channel pauses. 2. On return, <code>useFocusEffect</code> re-fetches and re-subscribes; cached data displays immediately while the refetch completes. A3. Group has no submissions today 1. The today’s-posts section shows an empty state encouraging someone to be first. 2. The still-to-post row shows everyone. 3. The missed-yesterday strip may also show everyone if yesterday was empty.
Exceptions	E1. Realtime connection drops 1. The Realtime channel disconnects. 2. The TanStack Query cache continues to serve cached data. 3. On reconnect (or next focus) the queries refetch and resync. 4. The user is not blocked from anything; only the live-update affordance pauses. E2. Trigger fails to update counters 1. Submission status flips to <code>approved</code> but the points/streak trigger errors. 2. The <code>submissions</code> UPDATE still fires; the feed updates. 3. The leaderboard does not reorder until the counter trigger is re-run (operational concern; observability via Supabase logs). E3. Member is removed from the group while the screen is open 1. The admin removes the user. 2. RLS hides subsequent <code>group_members</code> and <code>submissions</code> rows for the removed user. 3. The screen falls back to a “You’re no longer in this group” state on next refetch.
Priority	High
Frequency of Use	Multiple times per day per active member.
Business Rules	BR-1. Each verified submission awards exactly 1 point and increments the member’s <code>current_streak</code> by 1. BR-2. A day without a verified submission resets that member’s <code>current_streak</code> to 0 (Phase 5 cron does the actual reset). BR-3. Points and streaks are derived server-side via triggers; the client never computes them. BR-4. Streak math is keyed on <code>local_date</code> (group timezone), not wall-clock time. BR-5. The leaderboard is ranked by <code>points DESC</code>, then <code>current_streak DESC</code>. BR-6. The feed shows only the current local day’s approved submissions (not pending or rejected). BR-7. Tombstones surface yesterday’s misses, not the day before.
Special Requirements	SR-1. Realtime subscriptions are scoped per-group with <code>filter: group_id=eq.<id></code> to avoid cross-group bandwidth. SR-2. Subscriptions are torn down on screen blur to prevent leaks. SR-3. Leaderboard updates after an admin approval should be visible end-to-end within 3 seconds on Wi-Fi. SR-4. Media in the feed is fetched via short-lived signed URLs from the private <code>submissions</code> bucket.

Field	Content
Assumptions	- Group sizes (5–10) keep leaderboard rendering trivially fast; no virtualization needed. - Realtime is best-effort; the fallback (refetch on focus) is acceptable for an alpha. “Today” and “yesterday” are defined by the group’s IANA timezone, not the device clock.

UC-005 Receive Push Notifications and Trigger Daily Rollover

Field	Content
ID and Name	UC-005 Receive Push Notifications and Trigger Daily Rollover
Created By	Chris Kelamyan, Timothy Do
Date Created	May 8, 2026
Primary Actor	Group Member (push recipient); System (rollover initiator)
Secondary Actors	Group Admin, Expo Push Service, APNs / FCM, <code>pg_cron</code> , Supabase Edge Function
Description	This use case describes how the system makes the “missed day visible within hours” promise infrastructurally real. Members receive a pre-deadline reminder when they have not yet submitted, plus activity pings on member submissions and admin verdicts. Admins receive pings when a submission needs review. At each group’s local midnight, <code>pg_cron</code> fires <code>handle_daily_rollover()</code>, which finalizes the day, resets streaks for members without an approved submission, and drops a missed-day tombstone for them. Pushes route through <code>notifications_outbox</code> → <code>push-dispatch</code> edge function → Expo Push Service. Phase 5 closes the loop.
Trigger	Push: a domain event in Postgres (submission inserted, status updated, pre-deadline window reached). Rollover: <code>pg_cron</code> firing every 5 minutes.
Preconditions	PRE-1. The app runs as an EAS development or production build (Expo Go does not register push tokens on Android in SDK 55). PRE-2. The user has granted notification permission and an Expo push token has been upserted to <code>profiles.expo_push_token</code>. PRE-3. <code>pg_cron</code> and <code>pg_net</code> extensions are enabled on the Supabase project. PRE-4. A Supabase Database Webhook on <code>notifications_outbox</code> insert is wired to call the <code>push-dispatch</code> edge function.

Field	Content
Postconditions	Push: POST-1. A row exists in <code>notifications_outbox</code> describing the event. POST-2. The <code>push-dispatch</code> edge function POSTs to <code>https://exp.host/--/api/v2/push/send</code> and stamps <code>notifications_outbox.sent_at</code>. POST-3. The recipient device receives the push within 60 seconds (95th percentile). Rollover: POST-4. For each member with no approved submission for yesterday's <code>local_date</code>, <code>current_streak</code> is reset to 0. POST-5. A tombstone row appears in the next day's feed for the missed day. POST-6. A <code>notifications_outbox</code> row with <code>kind='missed'</code> is queued for that user. POST-7. <code>groups.last_rolled_date</code> is advanced to the new local date (idempotent).
Normal Flow	Push (member submits, admin gets a ping): 1. A member submits today's proof (UC-003). 2. The <code>submissions</code> INSERT trigger inserts a row into <code>notifications_outbox</code> with <code>kind='new_submission'</code> and the admin's <code>user_id</code>. 3. The Database Webhook fires on the outbox insert and POSTs to <code>push-dispatch</code>. 4. <code>push-dispatch</code> loads the admin's <code>expo_push_token</code> from <code>profiles</code>. 5. <code>push-dispatch</code> POSTs the payload to <code>https://exp.host/--/api/v2/push/send</code> (up to 100 messages per call). 6. Expo Push Service forwards to APNs (iOS) or FCM (Android). 7. <code>push-dispatch</code> stamps <code>notifications_outbox.sent_at = now()</code> on success. 8. The admin's device displays the notification. 9. The admin taps the notification; <code>addNotificationResponseReceivedListener</code> routes them to the review queue for the relevant group. Rollover: 10. <code>pg_cron</code> fires <code>handle_daily_rollover()</code> every 5 minutes. 11. The procedure computes the current <code>local_date</code> for each distinct timezone in <code>groups</code>. 12. For each group whose <code>last_rolled_date < current_local_date - 1</code>, it iterates members. 13. For each member with no approved submission for yesterday's <code>local_date</code>, the procedure sets <code>current_streak = 0</code> and inserts a <code>notifications_outbox</code> row with <code>kind='missed'</code>. 14. The procedure updates <code>groups.last_rolled_date</code> to the new local date. 15. The Database Webhook on <code>notifications_outbox</code> fires <code>push-dispatch</code> for each missed-day push.
Alternative Flows	A1. Pre-deadline reminder 1. A scheduled cron job runs N hours before each group's local midnight. 2. For each member with no submission yet today, it inserts a <code>notifications_outbox</code> row with <code>kind='reminder'</code>. 3. <code>push-dispatch</code> sends the push. A2. Admin approves a submission and the submitter gets a ping 1. The <code>submissions</code> UPDATE trigger fires on <code>status='approved'</code>. 2. It inserts a <code>notifications_outbox</code> row with <code>kind='reviewed'</code> for the submitter. 3. <code>push-dispatch</code> sends the push. A3. Token rotation 1. The OS issues a new Expo push token to the device. 2. <code>addPushTokenListener</code> on the client upserts the new token to <code>profiles.expo_push_token</code>. 3. Subsequent pushes use the new token.

Field	Content
Exceptions	E1. Push token marked <code>DeviceNotRegistered</code> <code>push-dispatch</code> (or a scheduled receipts-poller) reads receipts from <code>/--/api/v2/push/getReceipts</code>. - For tokens with <code>details.error === 'DeviceNotRegistered'</code>, the system nulls <code>profiles.expo_push_token</code>. - The system stops pushing to that device until a fresh token is registered. E2. <code>push-dispatch</code> fails (network or Expo error) - The function returns without stamping <code>sent_at</code>. <code>notifications_outbox</code> rows with <code>sent_at IS NULL</code> are picked up by a retry sweep. - The system retries with backoff. E3. DST boundary crossed during rollover - The current <code>local_date</code> computation uses Postgres <code>(now() AT TIME ZONE groups.timezone)::date</code>, which is DST-aware. - The rollover procedure does not double-process or skip a day. E4. Rollover procedure runs twice for the same day - The condition <code>last_rolled_date < current_local_date - 1</code> short-circuits the second run. - No streaks are double-reset; no duplicate tombstones are created. E5. User has not granted notification permission - The OS does not deliver pushes. - The Today screen always shows a self-computed countdown to the local-midnight cutoff so a missed reminder push does not by itself cause a missed submission.
Priority	High
Frequency of Use	Push: many per day per active group. Rollover: every 5 minutes (system); per-group rollover work runs once per local day.
Business Rules	BR-1. Push delivery is best-effort; the app does not rely on push as the only path to the user. BR-2. All pushes route through Expo Push Service; the app never calls APNs or FCM directly. BR-3. Rollover is idempotent — running it twice for the same day produces the same result. BR-4. Rollover is DST-safe — derived from IANA timezones in Postgres, not from offsets. BR-5. Each push corresponds to exactly one <code>notifications_outbox</code> row (single source of truth for delivery audit). BR-6. Push tokens are stored on the <code>profiles</code> row; a user with multiple devices is supported by re-registration on each device.
Special Requirements	SR-1. Push delivery target — 95% of pushes within 60 seconds of trigger. SR-2. Rollover sweep target — completes a single group's rollover in under 1 second. SR-3. The Database Webhook on <code>notifications_outbox</code> uses the project's <code>service_role</code>; <code>service_role</code> never appears in the mobile bundle. SR-4. The <code>push-dispatch</code> edge function authenticates against Expo using <code>EXPO_ACCESS_TOKEN</code> with Enhanced Security for Push Notifications enabled. SR-5. Notification deep-link routing covers the four kinds: <code>reminder</code> → Today, <code>new_submission</code> → group detail, <code>reviewed</code> → group detail, <code>missed</code> → Today.
Assumptions	- Real-device testing on iOS and Android 13+ is mandatory before friend-group rollout. - Expo Push Service's free tier is sufficient for alpha (well under 600 messages/sec/project). - APNs and FCM credentials are configured once via <code>eas credentials</code> and are stable thereafter.

3. Software Requirements Specification

3.1 Introduction

3.1.1 Purpose

This SRS specifies the functional and non-functional requirements for the Accountibuzz v1.0 MVP. It is the contract against which Phases 1 through 6 are planned, executed, and verified, and it is the bridge between the higher-level vision in `.planning/PROJECT.md` and the implementation artifacts in `app/`, `src/`, and `supabase/`. It is intended to be used by the project owner during design and code review, by the COMP586 instructor during evaluation, and by any future contributor who needs to understand what the product is supposed to do without reading the entire repository.

3.1.2 Document Conventions

- **Requirement IDs** follow the form `<AREA>--<NN>` with the area chosen from a fixed list: AUTH, GRP, INV, SUB, ADM, PTS, LB, FEED, NOTIF, ROLL, PLAT
- **Stable identifiers:** a deprecated requirement keeps its number, annotated `(superseded by <NEW-ID>)`
- **Phase IDs** are zero-padded two-digit numbers (`01`, `02`, ...); **plan IDs** are `<PHASE>--<NN>` (`02-07`, `03-01`)
- **JIRA issues** live in the `SCRUM` project at `comp586.atlassian.net`
- **Status markers:** `[ ]` open, `[x]` verified done, `(blocked: <reason>)` appended for blocked items
- **Code identifiers** (column names, RPC names, file paths) are written in `monospace`
- **Tables** in this document are descriptive, not runnable
- **Keywords** “must”, “should”, and “may” follow IETF RFC 2119 conventions

3.1.3 Project Scope

In scope:

- Mobile React Native application for iOS and Android via Expo SDK 55
- Supabase Postgres backend with Row-Level Security and SECURITY DEFINER RPCs
- Supabase Storage for media (avatars + submissions)
- Supabase Realtime for live leaderboard, feed, and admin queue updates
- Supabase Edge Functions (Deno) for push fan-out and other cross-user side-effects
- `pg_cron`-driven daily rollover for streak resets

Out of scope:

- Any web or desktop client
- OAuth, magic-link, or phone-OTP authentication
- Monetization or billing
- Public group discovery

- AI-assisted verification
- Financial commitment contracts
- In-app chat or DMs
- Per-user timezone handling (group timezone is the unit of accounting)

3.2 Overall Description

3.2.1 Product Perspective

Accountibuzz is a self-contained product, not a component of a larger system. It sits on top of three external platforms: Expo (build, OTA, push), Supabase (auth, database, storage, realtime, edge functions, cron), and Expo Push Service (delivery to APNs and FCM). It does not integrate with any third-party application data source. The system communicates with the outside world only through the user's device camera, the user's social graph via the OS share sheet for invite links, and the platform notification stacks. A single Postgres database is the system of record; the mobile client is a stateful cache and capture surface in front of it.

3.2.2 User Classes and Characteristics

User class	Description	Frequency of use	Technical sophistication
Member	Belongs to one or more groups, submits daily proof, sees the feed and leaderboard	Daily	Low — assumes a normal smartphone user
Admin	Created the group; reviews submissions and manages membership	Daily, in short bursts	Low to medium
First-time user	Has tapped an invite link or downloaded the app fresh; needs onboarding	Once per install	Low
Solo builder / maintainer	Project owner; uses the planning artifacts and migrations	Daily during a phase	High

3.2.3 Operating Environment

Surface	Environment
Mobile client	iOS 16+ and Android 13+, Expo SDK 55, React Native 0.83.1, React 19.2, Hermes v1, New Architecture

Surface	Environment
Backend	Supabase managed Postgres 15 + PostgREST + GoTrue + Storage + Realtime + Edge Runtime (Deno) + <code>pg_cron</code> , <code>pg_net</code> , <code>pgcrypto</code> , <code>uuid-oss</code> extensions
Build / distribution	EAS Build for native binaries; Expo Go for in-Phase iteration up to Phase 4; EAS dev/prod build from Phase 5 onward; TestFlight / Play Internal during alpha
Push delivery	Expo Push Service in front of APNs and FCM
Local dev	macOS, Node 20, Supabase CLI, EAS CLI, Xcode and Android Studio for native builds

3.2.4 Design and Implementation Constraints

Stack constraints (fixed):

- React Native via Expo SDK 55 (New Architecture only); Supabase as the all-in-one backend
- TypeScript for both client and database type generation
- Luxon for client-side timezone *display* only (never for business logic)
- TanStack Query v5 for server-state caching
- Zustand only for genuine client state, never for cached server data
- Zod + react-hook-form for input validation
- Manrope variable font + Feather icons for the design system

Schema and data constraints:

- Migrations are append-only: once `0001_foundation.sql` shipped, every change is a new file (`0002_*`, `0003_*`, ...)
- Every public-schema table must have RLS enabled, enforced in CI by `rls-check.yml`
- The `local_date` on submissions is derived server-side from the group's IANA timezone and never sent by the client
- The submissions table carries a unique `(group_id, user_id, local_date)` constraint as the canonical defense against double-counting
- Streak math runs in a trigger off the `submissions.status → 'approved'` transition; the client never computes it

Pipeline and product constraints:

- Submissions use a two-phase commit: storage upload first, row insert second, with cleanup of orphans
- Single-admin trust model is fixed; multi-admin and rotating admin are out of scope for v1

3.2.5 Assumptions and Dependencies

- A cooperative friend-group context where social pressure is enough incentive — no financial stakes
- Users will install an EAS build (TestFlight or Play Internal) during alpha
- Apple and Google continue to allow Expo's push pipeline to operate
- Supabase continues to host `pg_cron` and edge functions on the project's tier
- The IANA timezone database remains the right authority for "local midnight" semantics

3.3 System Features

This section describes the v1 feature set. Each feature lists its purpose, its functional requirements (mapped to requirement IDs from `REQUIREMENTS.md`), and the relevant phase. Status reflects the state of the project as of 2026-05-08: Phases 1–3 are complete; Phase 4 is planned and the UI contract approved; Phases 5–6 are pending.

3.3.1 Authentication and Profile

Description. Bring users into the app, persist their session across restarts, let them sign out, and let them set a display name and avatar. This is the identity layer everything else depends on.

Functional Requirements.

ID	Requirement	Status
AUTH-01	A user can sign up for an account on a mobile device using email and password (Supabase Auth)	Done
AUTH-02	A user can sign in and the session persists across app restarts via the AsyncStorage adapter wrapped by SecureStore-encrypted key	Done
AUTH-03	A user can sign out from the app, which clears the session and the React Query cache	Done
AUTH-04	A user can create or edit a profile with display name and avatar (uploaded to the <code>avatars</code> storage bucket, path-gated to the user's own UUID)	Done
PLAT-01	The app runs on iOS and Android via Expo SDK 55 (iOS verified via UAT; Android deferred until EAS dev-build phase)	Partial
PLAT-02	A CI workflow (<code>rls-check.yml</code>) fails the build if any public-schema table has Row-Level Security disabled	Done

3.3.2 Groups and Invites

Description. Form the container for accountability. A user creates a group with a name, goal, submission type, and timezone, becomes its admin, generates a shareable code, and invites friends. Membership is capped at ten.

Functional Requirements.

ID	Requirement	Status
GRP-01	A user can create a group with name, goal, submission type (photo or video), and IANA timezone	Done
GRP-02	The group creator becomes the admin of that group	Done
GRP-03	A user can view the list of groups they belong to	Done
GRP-04	A user can view a group's details, members, and rules	Done
GRP-05	A user can leave a group (admins must transfer first)	Done

ID	Requirement	Status
INV-01	An admin can generate a shareable invite link or 8-character code (31-char ambiguity-stripped alphabet)	Done
INV-02	A user can join a group by tapping an invite link or entering a code	Done
INV-03	Group membership is capped at 10 (enforced under row lock to prevent TOCTOU)	Done

3.3.3 Capture and Admin Review

Description. The core daily loop. Members submit a photo or video of their commitment before the group's local-midnight cutoff. The admin reviews each submission. A rejection is terminal – the submitter does not get to re-submit that day. Uploads are resilient to flaky networks via two-phase commit and an offline queue.

Functional Requirements.

ID	Requirement	Status
SUB-01	A member can capture and submit a photo (if the group is photo-type) before local midnight	Done
SUB-02	A member can capture and submit a short video (if the group is video-type) before local midnight	Done
SUB-03	Upload is resilient to flaky networks (two-phase commit, AsyncStorage offline queue, retry)	Done
SUB-04	A member can see the status of their submission (pending / approved / rejected)	Done
SUB-05	A member is blocked from submitting twice on the same local day (unique constraint)	Done
SUB-06	A member can optionally add a short caption (≤ 140 chars)	Done
ADM-01	An admin sees a queue of pending submissions for groups they admin (<code>get_pending_review_queue</code> RPC)	Done
ADM-02	An admin can approve a submission	Done
ADM-03	An admin can reject a submission with an optional reason (≤ 140 chars)	Done
ADM-04	A rejected submitter is notified that today did not count; rejection is terminal	Done
PLAT-03	Group admin can only approve or reject submissions for groups they admin (RLS + RPC double-gate)	Done

3.3.4 Points, Streaks, Leaderboard, and Feed

Description. Convert verified submissions into visible social signal. One point per verified day, streak increments on consecutive verified local dates, streak resets to zero on any miss. The leaderboard ranks by points then streak and updates in near real time. The group feed surfaces today's approved posts, who has not yet posted, and yesterday's misses.

Functional Requirements.

ID	Requirement	Status
PTS-01	Each verified submission awards 1 point and increments the member's streak in that group	Pending (Phase 4)
PTS-02	A day without a verified submission resets that member's streak to zero	Pending (Phase 5 cron)
PTS-03	Streaks and points are derived server-side and consistent with the group timezone	Pending
LB-01	A group has a leaderboard ranked by total points, with each member's current streak visible	Pending
LB-02	The leaderboard updates in near real time as submissions are approved (Supabase Realtime → TanStack Query cache patch)	Pending
FEED-01	Group members can see today's approved submissions for their group	Pending
FEED-02	The feed shows which members have not yet submitted today	Pending
FEED-03	The feed shows missed-day tombstones for yesterday's misses	Pending

3.3.5 Notifications and Daily Rollover

Description. Make the “missed day visible within hours” promise infrastructurally real. Pre-deadline reminder, activity pings on member submissions and admin verdicts, and a `pg_cron` job that rolls each group at its own local midnight in a DST-safe way.

Functional Requirements.

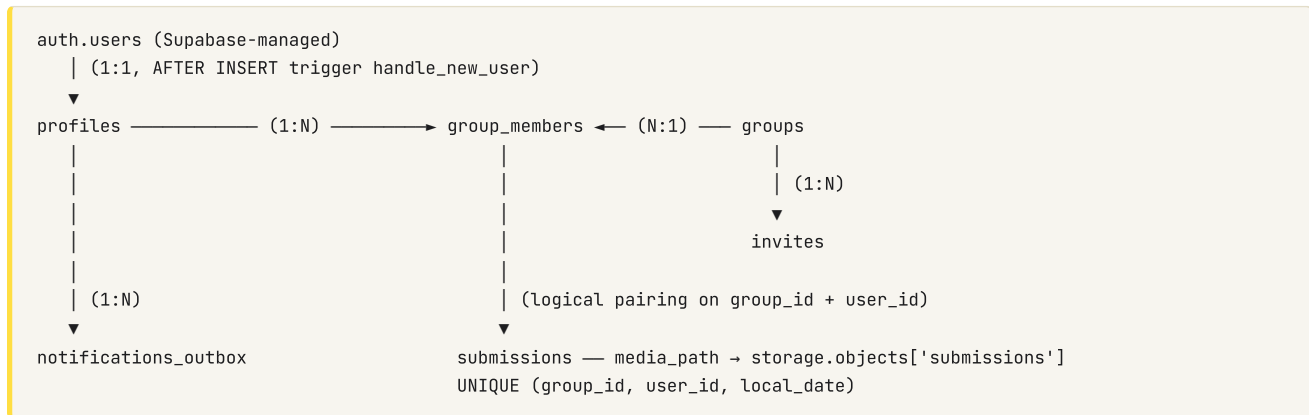
ID	Requirement	Status
NOTIF-01	A member receives a push reminder before the group's daily cutoff if they have not submitted	Pending (Phase 5)
NOTIF-02	A member receives a push when another member submits	Pending
NOTIF-03	An admin receives a push when a submission needs review	Pending
NOTIF-04	A member receives a push when the admin approves or rejects their submission	Pending

ID	Requirement	Status
NOTIF-05	The app handles push on both iOS and Android via an EAS dev or production build	Pending
ROLL-01	At each group's local midnight, the system finalizes the day; members without an approved submission get their streak reset and a tombstone created	Pending
ROLL-02	Rollover is idempotent and DST-safe	Pending

3.4 Data Requirements

3.4.1 Logical Data Model

The schema lives entirely in the `public` schema of a single Postgres database, with primary keys as `uuid` (default `gen_random_uuid()`) and timestamps as `timestampz default now()`. Storage objects live in two buckets, `avatars` (public-read) and `submissions` (private). The relationships are:



The unique `(group_id, user_id, local_date)` constraint on `submissions` is the canonical guard against double-submission, and it is enforced at the database, not at the client. The `local_date` value is derived inside the `submit_today` RPC from `(now() AT TIME ZONE groups.timezone)::date`, so the calendar date a submission counts for is decided by the database against the group's IANA timezone — the client never sends it. Counter columns on `group_members` (`points`, `current_streak`, `longest_streak`, `last_rolled_date`) are mutated by the `handle_submission_approval` trigger (Phase 4 body) and by `handle_daily_rollover` (Phase 5 body), never by the client.

3.4.2 Data Dictionary

Table	Column	Type	Notes
<code>profiles</code>	<code>id</code>	uuid PK	FK to <code>auth.users(id)</code> ON DELETE CASCADE
	<code>display_name</code>	text not null	Default <code>''</code> ; user sets during onboarding

Table	Column	Type	Notes
	avatar_path	text	Storage path inside avatars bucket
	created_at , updated_at	timestampz	
groups	id	uuid PK	
	name	text not null	CHECK 1-60 chars
	goal	text not null	CHECK 5-140 chars
	submission_type	text not null	CHECK in (photo , video)
	timezone	text not null	IANA identifier, e.g. America/Los_Angeles
	admin_user_id	uuid not null	FK to profiles(id) ON DELETE RESTRICT
	created_at	timestampz	
group_members	(group_id, user_id)	composite PK	FKs cascade from groups / profiles
	role	text not null	CHECK in (member , admin); default member
	joined_at	timestampz	
	points	int not null default 0	Trigger-maintained
	current_streak	int not null default 0	Trigger-maintained
	longest_streak	int not null default 0	Trigger-maintained
	last_rolled_date	date	Group-local date of last verified or rolled day
submissions	id	uuid PK	
	group_id , user_id	uuid not null	FKs cascade
	local_date	date not null	Server-derived from group timezone
	status	text not null	CHECK in (pending , approved , rejected); default pending
	caption	text	≤ 140 chars
	media_path	text not null	Storage path inside submissions bucket

Table	Column	Type	Notes
	<code>media_type</code>	text not null	CHECK in (<code>photo</code> , <code>video</code>)
	<code>reviewed_by</code>	uuid	FK to <code>profiles(id)</code>
	<code>reviewed_at</code>	timestampz	
	<code>rejection_reason</code>	text	≤ 140 chars
	<code>created_at</code>	timestampz	
	UNIQUE	(<code>group_id</code> , <code>user_id</code> , <code>local_date</code>)	Pitfalls §2 race guard
<code>invites</code>	<code>id</code>	uuid PK	
	<code>group_id</code>	uuid not null	FK cascades
	<code>code</code>	text unique not null	8-char, 31-char alphabet
	<code>created_by</code>	uuid not null	FK to <code>profiles(id)</code>
	<code>expires_at</code>	timestampz	Default <code>now() + 7 days</code>
	<code>used_at</code> , <code>used_by</code>	timestampz, uuid	Stamped on consume
	<code>created_at</code>	timestampz	
<code>notifications_outbox</code>	<code>id</code>	uuid PK	
	<code>user_id</code>	uuid not null	Recipient
	<code>kind</code>	text not null	One of <code>reminder</code> , <code>new_submission</code> , <code>reviewed</code> , <code>missed</code>
	<code>payload</code>	jsonb not null default <code>'{}'::jsonb</code>	Routing data for deep link
	<code>created_at</code> , <code>sent_at</code>	timestampz	<code>sent_at IS NULL</code> means undelivered

3.4.3 Reports

The MVP does not expose user-facing reporting beyond the leaderboard and feed described in Section 3.3.4. Internal reports for project management are derived from JIRA and from the planning artifacts: per-phase progress from `ROADMAP.md` checkboxes, sprint burndown from JIRA's burndown chart, and a categorical requirements distribution exported from JIRA to `docs/charts/requirements_distribution_jira.png`. A traceability matrix linking each requirement ID to phase, plan, JIRA epic, JIRA stories, and status is maintained in `docs/processes/04-tracing.md` and regenerated on each phase close.

3.4.4 Data Integrity, Retention, and Disposal

Integrity (enforced at the database level):

- Foreign keys on `group_members`, `submissions`, and `invites` cascade deletes from `groups` — removing a group sweeps all its children
- Profile deletion cascades to membership and submission rows because the user owns those rows
- CHECK constraints on `groups.name`, `groups.goal`, `groups.submission_type`, and `submissions.status` reject malformed rows at insert time
- The unique `(group_id, user_id, local_date)` constraint on `submissions` prevents duplicate-day submissions even under concurrent client retries
- RLS policies prevent cross-group reads and writes
- The `submissions_select_group_members` storage policy gates media reads to the same group-member check used on the table

Retention (intentionally simple for the MVP):

- Submissions and their media are kept for the life of the group
- Deleting a group cascades all rows; a scheduled cleanup job (planned in Phase 6) removes orphaned storage objects whose `media_path` no longer matches any `submissions.media_path`
- A user who deletes their account cascades deletion of their profile, membership rows, and submission rows
- Push tokens are nulled on `DeviceNotRegistered` returns from the Expo Push receipts endpoint
- Backups are whatever Supabase provides on the project’s tier; no independent backup pipeline during alpha

Disposal of credentials:

- The `service_role` key is never present in the mobile bundle
- The `anon` key is treated as public — it is in the bundle, extractable in seconds, and protected by RLS rather than secrecy
- Old invite codes are marked used (not deleted), preserving an audit trail

3.5 External Interface Requirements

3.5.1 User Interfaces

The app uses a hand-rolled React Native design system (no shadcn, since shadcn is web-only) defined by `src/theme/tokens.ts` and `ThemeProvider`.

Property	Value
Primary	<code>#FFDE42</code> (yellow)
Accent	<code>#53CBF3</code> (cyan)
Destructive	<code>hsl(4 78% 56%)</code>
Success	<code>hsl(145 55% 42%)</code>
Theme	Warm off-white / near-black dual theme; ships both light and dark modes
Type family	Manrope variable
Type sizes	5 only — Display 32 / H1 24 / H2 20 / Body 16 / Caption 13

Property	Value
Type weights	3 only — 500 Medium / 700 Bold / 800 ExtraBold
800 weight scope	Reserved for Display, with one documented exception on the leaderboard points number
Spacing scale	4-pt grid: 4 / 8 / 12 / 16 / 24 / 32
Radii	6 / 12 / 20 + pill
Elevations	<code>e1-subtle</code> , <code>e2-raised</code> ; 1-pixel border in dark mode
Touch target minimum	44 pt (iOS HIG)

Voice rules:

- The word `Cancel` is banned as a dismiss label; every modal supplies a context-specific dismiss
- Prefer first-person plural copy (`we` , `your group` , `friends`) over institutional copy (`user` , `organization`)

Component primitives by phase (full anatomy in `01-UI-SPEC.md` ... `04-UI-SPEC.md`):

- **Phase 1** — 11 primitives: buttons, inputs, modals, avatars, screen scaffolding
- **Phase 2** — segmented control, invite chip, IANA timezone picker
- **Phase 3** — camera stack: Shutter, CaptureTopBar, ReviewPanel, SwipeCard, StatusPill, TypeChip
- **Phase 4** — LeaderboardRow, FeedItem, StillToPostAvatarRow, plus an inline `MissedYesterdayTombstones` pattern

Lovable mockups checked into `design_refs/` are the screen-layout source of truth for each phase; tokens always defer to Phase 1.

3.5.2 Software Interfaces

Interface	Purpose	Protocol / library	Notes
Supabase Auth (GoTrue)	Account creation, session management, password reset	HTTPS via <code>@supabase/supabase-js</code> v2.58	Email + password; <code>detectSessionInUrl: false</code> for RN; <code>AppState</code> listener toggles <code>startAutoRefresh</code> / <code>stopAutoRefresh</code>
Supabase PostgREST	Reads on tables and RPC calls	HTTPS via <code>supabase-js</code> , JWT in header	All writes go through SECURITY DEFINER RPCs; reads use RLS-gated table queries
Supabase Realtime	Live leaderboard, live feed, live admin queue	WSS via <code>supabase-js</code> channel API	Channel scoped per group, torn down on screen blur to avoid leaks

Interface	Purpose	Protocol / library	Notes
Supabase Storage	Media (submissions) and avatars	HTTPS multipart upload via <code>supabase-js</code>	<code>submissions</code> is private; <code>avatars</code> is public-read; signed URLs for media reads
Supabase Edge Functions	Push fan-out, future moderation	HTTPS POST from Database Webhooks; Deno runtime	<code>push-dispatch</code> and a future <code>daily-rollover-helper</code> are the only ones in v1
Expo Push Service	Push delivery to APNs and FCM	HTTPS POST to <code>https://exp.host/--/api/v2/push/send</code>	Up to 100 messages per call; receipts polled to detect dead tokens
Expo Camera	Photo and video capture	<code>expo-camera</code> SDK 55 (<code>CameraView</code> + <code>recordAsync</code>)	Permission required; no camera-roll fallback by default
Expo Linking	Deep links for invites and notification taps	<code>expo-linking</code> + Expo Router file routes	Universal links (iOS AASA + Android assetlinks) plus <code>accountibuzz://</code> scheme fallback
OS share sheet	Sharing invite links	<code>expo-clipboard</code> and platform share	Used by <code>shareInvite</code>

3.5.3 Hardware Interfaces

The app touches three pieces of device hardware. The camera (front or rear, user-selectable inside the capture flow) is accessed through `expo-camera`'s `CameraView` and is required for the core submission flow. Local storage holds the encrypted session blob (`@react-native-async-storage/async-storage` wrapped with a 256-bit AES key in `expo-secure-store`), the offline submission queue, and the cached pending invite. The notification subsystem is reached through `expo-notifications`, which registers an Expo push token with the Expo Push Service. The device's APNs (iOS) and FCM (Android) credentials are configured once during EAS build and never touched at runtime.

3.5.4 Communication Interfaces

- All client-server traffic is over HTTPS or WSS
- The supabase-js client is configured with the project URL and the public `anon` key from `app.config.ts`; it authenticates user requests with the JWT obtained from Supabase Auth
- Realtime subscriptions use a WSS channel per active group, scoped by `filter: group_id=eq.<id>`, and are torn down on screen blur
- Outbound push goes through Expo Push Service, never directly to APNs or FCM
- Database webhooks fire on `notifications_outbox` insert and POST to the `push-dispatch` edge function
- The user's device never communicates peer-to-peer with another device; every message routes through Postgres or Storage

3.6 Quality Attributes

3.6.1 Usability Requirements

- **Capture in 3 taps:** From the Today screen, the dominant case completes in Capture → Shutter → Submit
- **Immediate pending state:** The Pending status pill must appear the moment Submit is tapped, with upload progressing in the background. A delayed-by-the-admin Pending state is what makes the submitter feel the loop is real
- **Context-specific dismiss labels:** Modal dismiss buttons are verbs (“Keep editing”, “Discard draft”), never the word `Cancel`, so the user always knows what they are about to abandon
- **Two-layer validation with plain-language copy:** Every form field validates client-side via Zod and re-validates server-side via typed RPC errors. Copy names the failure plainly (“This invite is already used”) rather than echoing a technical code
- **Recipient-may-not-have-the-app deep linking:** Universal-link landing routes through the App Store / Play Store and replays the pending invite after install
- **WCAG 2.1 AA color contrast** on text against background, at the chosen palette (verified at design time in `01-UI-SPEC.md`)
- **44 pt minimum touch target** on every interactive element (iOS HIG)

3.6.2 Performance Requirements

Metric	Target
Cold-start to Today screen	≤ 2 seconds on a mid-tier 2024 phone over Wi-Fi
Capture screen ready (camera preview live)	≤ 1 second from tap
Submission upload — photo on a stable Wi-Fi connection	≤ 5 seconds for an 80% JPEG at default resolution
Submission upload — video, capped at 15 seconds, 720p, ~2 Mbps	≤ 20 seconds on stable LTE
Leaderboard refresh after admin approval (Realtime)	≤ 3 seconds end-to-end on Wi-Fi
Admin queue load time (<code>get_pending_review_queue</code>)	≤ 500 ms server time at 50-row cap
Push notification delivery (trigger to device)	≤ 60 seconds, 95th percentile
Daily rollover sweep	Runs every 5 minutes, completes a single group's rollover in under 1 second

The system is tuned for groups of 5–10 with a few dozen active users in alpha; scaling beyond a few thousand users would invite the considerations in `ARCHITECTURE.md §Scaling`.

3.6.3 Security Requirements

Row-Level Security is the single authorization boundary. Every public-schema table has RLS enabled, and a CI workflow (`rls-check.yml`) fails the build if a new table ever ships without it. The `service_role` key never enters the mobile bundle; it lives only in edge functions and CI secrets. The `anon` key is treated as public — it is in the binary, extractable in seconds, and protected entirely by RLS. Policies reference `auth.uid()` and joins to `group_members`, never `auth.jwt()—>'user_metadata'`, because end users can mutate their own metadata. SECURITY DEFINER functions pin `set search_path = public` to prevent search-path attacks. Cross-group attacks on review actions are blocked by re-resolving the submission's `group_id` from the database before checking admin status. Storage policies on the private `submissions` bucket mirror the table policy, gating reads by `is_group_member`; ex-members lose access immediately and signed URLs are short-lived. Invite codes use a 31-character ambiguity-stripped alphabet, are 8 characters long (about 8.5×10^{11} possibilities), expire by default after 7 days, and `get_invite_preview` returns a uniform `invite_not_found` for any non-matching code so attackers cannot enumerate state. Pgcrypto provides UUID generation; rate limiting on submission insert is provided by the unique constraint plus a planned per-user rate limit at the edge.

3.6.4 Safety Requirements

The system handles user-generated photo and video content; it does not control physical equipment, dispense medication, or operate vehicles, so traditional safety-critical analysis does not apply. The relevant safety concern is content harm. The product mitigates this by (a) keeping all groups invite-only with no public discovery, so the social context is bounded to people who already know each other, (b) gating media reads to current group members, with ex-members losing access immediately, and © keeping media in a private storage bucket behind short-lived signed URLs. An admin can reject a submission with a reason and can remove a member; a member can leave a group at any time. There is no in-app harassment surface (no DMs, no chat, no public profiles), which limits the abuse vectors a small social product typically inherits. If a user deletes their account, all their submissions and memberships cascade-delete, removing the user's content from the system without manual intervention.

3.6.5 Availability Requirements

The MVP targets best-effort availability appropriate for an alpha. The mobile client is offline-tolerant on the read path (cached feed and leaderboard) and offline-tolerant on the write path for submissions (AsyncStorage queue retries on reconnect). The backend's availability is whatever Supabase provides on the project's plan; the project does not run an independent failover. Push notifications are best-effort by design. Expo Push Service does not guarantee delivery, and the app does not rely on push as the only path to the user. The Today screen always shows a self-computed countdown to the local-midnight cutoff so that a missed reminder push does not by itself cause a missed submission. A scheduled job runs the daily rollover every five minutes, which keeps streak-reset latency bounded even when a single run fails.

3.6.6 Robustness Requirements

The system is designed to fail predictably under the conditions a mobile app actually encounters.

Failure mode	Mitigation
Network drops mid-upload	Two-phase commit pipeline — storage upload first, retried on failure without producing an orphaned <code>submissions</code> row

Failure mode	Mitigation
Storage upload succeeds but <code>submit_today</code> RPC fails	Planned cleanup job sweeps storage objects with no matching <code>submissions.media_path</code>
User taps Submit twice on flaky network	Unique <code>(group_id, user_id, local_date)</code> constraint; second attempt raises <code>already_submitted_today</code> , treated as the same final state
Two admins approve the same submission concurrently	<code>for update</code> row lock + <code>where status = 'pending'</code> clause inside <code>review_submission</code> ; loser deterministically receives <code>not_pending</code>
Admin of group A attempts to review a submission belonging to group B	<code>review_submission</code> re-resolves <code>group_id</code> from the database before checking admin status
Realtime subscriptions leak as screens unmount	Channel teardown on screen blur; battery cost stays bounded
Auth refresh hammers the endpoint while app is backgrounded	<code>AppState</code> listener calls <code>stopAutoRefresh()</code> on background, <code>startAutoRefresh()</code> on foreground
JWT expires mid-upload	supabase-js client auto-refreshes on next request boundary; queue manager retries long uploads that span an expiry
Push token goes stale on device	Tokens marked <code>DeviceNotRegistered</code> by Expo's receipts endpoint are nulled out so the system stops hitting them
Production schema diverges from source	Migrations are append-only — no edited migration ever ships — so replaying <code>supabase/migrations/</code> in order always rebuilds the live schema

Last updated: 2026-05-08 Source artifacts: `.planning/PROJECT.md`, `.planning/REQUIREMENTS.md`, `.planning/ROADMAP.md`, `.planning/STATE.md`, `.planning/research/` (STACK / ARCHITECTURE / FEATURES / PITFALLS / SUMMARY), `.planning/phases/01-foundation/`, `02-groups-invites/`, `03-capture-admin-review/`, `04-social-surfaces/`, `docs/processes/01-04`, `supabase/migrations/0001..0007`, `app/`, `src/`, `package.json`